1 Цель работы

Изучить вызов процедур на языке ассемблера из C++ и передачу данных между модулями программы.

2 Постановка задачи

Вариант 7. Написать процедуру на языке ассемблера, которая по исходной строке формирует палиндром с использованием цепочечных команд. Ввод исходной строки и вывод результата выполнить в C++. Данные передавать через аргументы функции по соглашению `cdecl`.

К исходной строке S дописывается её обратная запись S^R. Полученная строка $P = S\,S^R$ читается одинаково в обоих направлениях. Например, `abcd` преобразуется в `abcddcba`.

Программа предназначена для Windows, платформа x86, сборка MSVC и MASM. Допускается ввод до 255 байтов в кодировке CP866. Русская буква в этой кодировке занимает один байт. Пробелы, регистр и знаки препинания сохраняются. Ввод большей длины отклоняется. Для результата выделен буфер из 511 байтов: 510 символов и завершающий нуль.

3 Алгоритм

1. В модуле C++ прочитать строку с помощью `std::getline`. Проверить успешность ввода и длину n. Кодировка консоли — CP866.

2. Передать ассемблерной процедуре адрес исходной строки, адрес буфера результата и n. Сохранить используемые регистры `EBP`, `ESI` и `EDI`.

3. Сбросить флаг направления командой `CLD`. Загрузить адреса строк в `ESI` и `EDI`, а n в `ECX`. Скопировать исходную строку командой `REP MOVSB`.

4. Если $n = 0$, пропустить обратное копирование. Иначе установить `ESI` на последний символ исходной строки и выполнить n повторений: `STD; LODSB; CLD; STOSB`. Чтение идёт назад, запись вперёд.

5. Записать нулевой байт в конец результата, восстановить регистры и вернуться в C++ командой `RET`. Вывести полученную строку.

4 Интерфейс модулей и используемые команды

В C++ процедура объявляется с `extern "C"`. Соглашение о вызове задаётся словом `__cdecl`. В MASM используется имя `_MakePalindrome`, экспортируемое директивой `PUBLIC`. Результат записывается по переданному адресу.

Аргументы помещаются в стек справа налево. После `PUSH EBP; MOV EBP, ESP` по адресу `[EBP+8]` находится адрес исходной строки, по `[EBP+12]` — адрес результата, по `[EBP+16]` — длина строки. Аргументы из стека удаляет вызывающий код C++, поэтому используется `RET` без операнда.

`REP MOVSB` копирует байты, `LODSB` читает байт в `AL`, `STOSB` записывает его в результат. `STD` задаёт движение указателя назад, `CLD` — вперёд. Перед возвратом флаг направления сброшен. `EBX` не изменяется, `EBP`, `ESI` и `EDI` сохраняются в стеке.

5 Тестирование

Сборка выполнена скриптом `code/build.bat` с инструментами Visual Studio 2026: модуль `main.cpp` компилируется MSVC x86, а `palindrome.asm` — MASM. При компоновке задан ключ `/MACHINE:X86`. Журнал сборки сохранён в `tests/program-build.log`.

В таблице приведены исходные данные и полученные результаты. В строках 8, 9 и 12 каждый пробел показан знаком ␣; других пробелов внутри этих строк нет. Степень обозначает число повторений.

№	Исходная строка	Результат	Код выхода
1	Пустая строка	Пустая строка	0
2	a	aa	0
3	ab	abba	0
4	abc	abccba	0
5	abcd	abcddcba	0
6	aba	abaaba	0
7	aaaa	aaaaaaaa	0
8	a␣b	a␣bb␣a	0
9	␣a␣	␣a␣␣a␣	0
10	Ab1!	Ab1!!1bA	0
11	тест	тесттсет	0
12	a␣роза	a␣розаазор␣a	0
13	$(\mathtt{abc})^{85}$, 255 символов	$(\mathtt{abc})^{85}(\mathtt{cba})^{85}$, 510 символов	0
14	$\mathtt{a}^{256}$	Ошибка: превышена длина ввода	1
15	$\mathtt{b}^{300}$	Ошибка: превышена длина ввода	1
16	1 пробел	2 пробела	0
17	3 пробела	6 пробелов	0
18	Нет строки (EOF)	Ошибка ввода	1

Скрипт `tests/run_tests.py` проверяет 18 случаев из таблицы и по одной строке каждой длины от 0 до 255 байтов. Он сравнивает стандартный вывод, сообщения об ошибках и код завершения с ожидаемыми. Все 274 случая прошли. Протокол каждого запуска находится в `tests/results.json`.

Отдельная программа `tests/check_abi.cpp` проверяет вызов процедуры для длин 0–255. Например, перед вызовом в `ESI` записывается `55667788h`, после возврата значение сравнивается с исходным. Аналогично проверяются `EBX`, `EDI`, `EBP` и `ESP`; флаг направления проверяется через `PUSHFD`. Также сравниваются результат, завершающий нуль и контрольные байты непосредственно до и после выходной строки. Все 256 проверок прошли; вывод сохранён в `tests/build.log`.

На рисунке 1 показан запуск с русской строкой из теста 11.

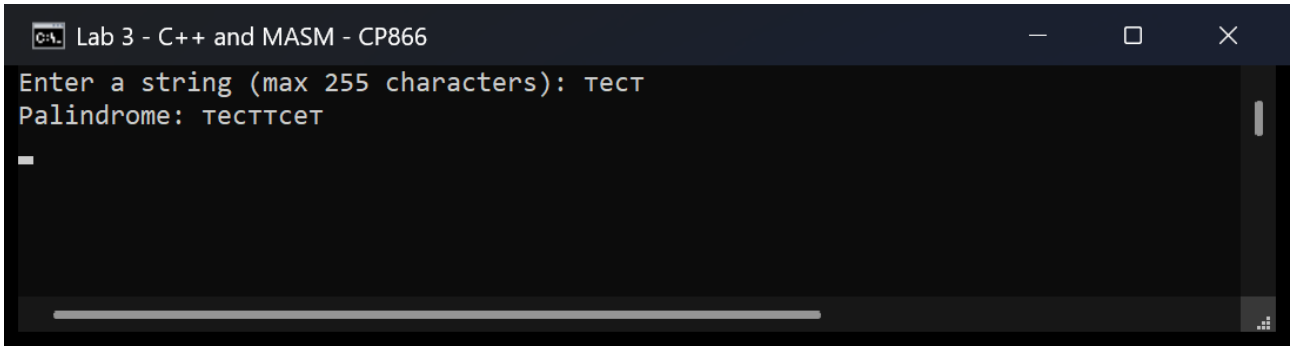


Рисунок 1 — Ввод и вывод русской строки в CP866

6 Код программы

Модуль C++ — main.cpp

```
#include <iostream>
#include <string>
#include <windows.h>

extern "C" void __cdecl MakePalindrome(
    const char* source, char* result, int length);

int main() {
    SetConsoleCP(866);
    SetConsoleOutputCP(866);

    const int MAX_LENGTH = 255;
    std::string source;
    char result[2 * MAX_LENGTH + 1];

    std::cout << "Enter a string (max 255 characters): ";
    if (!std::getline(std::cin, source)) {
        std::cerr << "Error: input failed.\n";
        return 1;
    }
    if (source.size() > MAX_LENGTH) {
        std::cerr << "Error: string is longer than 255 characters.\n";
        return 1;
    }

    MakePalindrome(source.c_str(), result, static_cast<int>(source.size()));
    std::cout << "Palindrome: " << result << '\n';
    return 0;
}
```

Модуль MASM — palindrome.asm

```
.386
.MODEL FLAT
PUBLIC _MakePalindrome

.CODE
; MakePalindrome(const char* source, char* result, int length), cdecl.
_MakePalindrome PROC
    PUSH EBP
    MOV EBP, ESP
    PUSH ESI
    PUSH EDI

    ; Копируем исходную строку в начало результата.
    CLD
    MOV ESI, [EBP+8]
    MOV EDI, [EBP+12]
    MOV ECX, [EBP+16]
    REP MOVSB

    ; Читаем исходную строку с конца и дописываем её копию.
    MOV ECX, [EBP+16]
    JECXZ palindromeReady
    MOV ESI, [EBP+8]
    ADD ESI, ECX
    DEC ESI
copyBackwards:
    STD
    LODSB
    CLD
    STOSB
    LOOP copyBackwards

palindromeReady:
    MOV BYTE PTR [EDI], 0
    POP EDI
    POP ESI
    POP EBP
    RET
_MakePalindrome ENDP
END
```

7 Вывод

Для вызова ассемблерной процедуры из C++ использованы три аргумента: адрес исходной строки, адрес результата и длина. При `cdecl` вызывающий код освобождает 12 байтов аргументов, поэтому в процедуре достаточно `RET` без операнда. Регистры `ESI` и `EDI`, используемые цепочечными командами, восстанавливаются перед возвратом. Сочетание `STD; LODSB; CLD; STOSB` позволяет читать строку с конца и одновременно записывать её обратную копию вперёд.